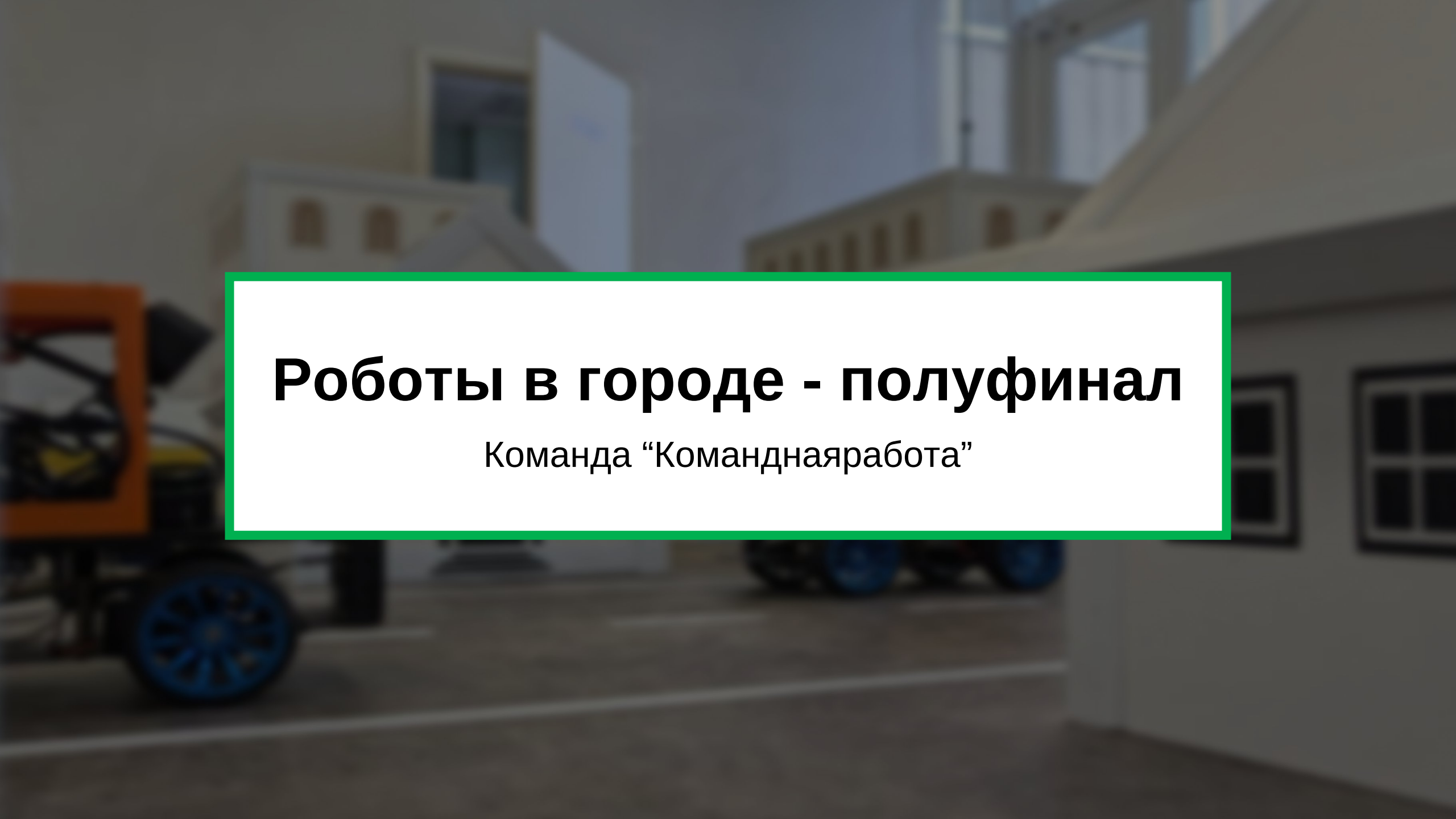


# Роботы в городе - полуфинал

Команда “Командняяработа”

# Формулировка задания

**Задача эстафеты** формулируется следующим образом:

- Робот под номером 1 должен доехать до робота номер 2 и сообщить ему об этом через conections.txt
- Робот под номером 2 должен доехать до робота номер 3 и сообщить ему об этом через conections.txt
- Робот под номером 3 должен доехать до финиша (Это одна из 15 пронумерованных точек, хранится в переменной Task)



Начальные положения роботов, номер точки финиша и координаты барьеров (для студенческого трека) выбираются случайным образом.

Участники присылают на проверку только скрипты agent_1_controller.py, agent_2_controller.py, agent_3_controller.py. Всё остальное будет проигнорировано организаторами

Траектория Кода

Выполнение задания происходиит в программе Траектория кода 1.2.0.

# Алгоритмы поведения роботов

Каждый из роботов (агентов) на поле в процессе выполнения своей задачи имеет перечень возможных состояний:

- **SEND_BEGIN_POS: Начальная рассылка координат -** агент транслирует всем остальным роботам свои стартовые координаты. 1-ый агент ничего не рассылает.
- **WAIT_REGIM: Режим ожидания** - агент приостанавливает активные действия и ожидает завершения манёвра предыдущего агента. 1-ый агент никогда не ждет.
- **GET_AGENTS_POS: Сбор данных о местоположении -** агент запрашивает и актуальные координаты всех остальных агентов для формирования маршрута.
- **MAIN_REGIM: Основной режим** - агент прокладывает путь к целевой точке, учитывая препятствия, барьеры и агентов и осуществляет движение по этому маршруту. По условиям задачи активным может быть только один агент, остальные на время его маневров статичны.
- **SEND_FINAL_POS: Рассылка финальных координат** - после успешного достижения цели агент уведомляет следующего о своих окончательных координатах. Самый последний (3-ий) агент никогда не пишет свои окончательные координатыы.
- **END: Останов** - финальное состояние, агент завершает выполнение своей программы и останавливается, становясь статичным препятствием на карте.

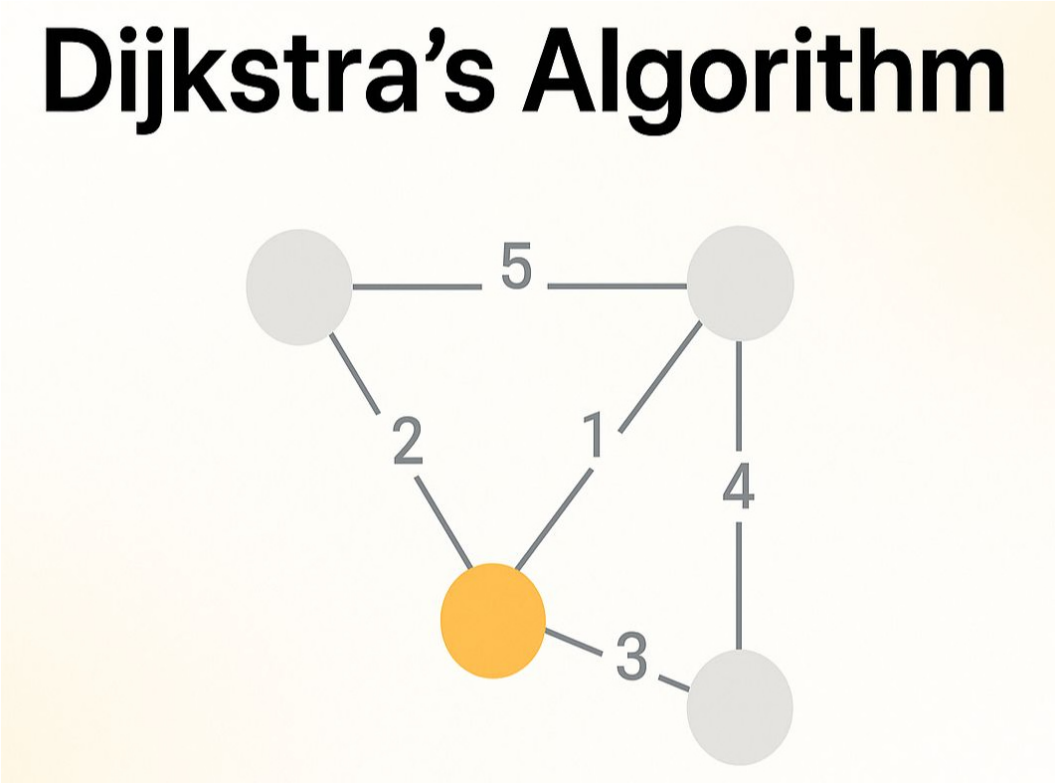
Агенты общаются через единый поток сообщений (файл conections.txt).

```
Agent2|Agent1|mypos|943.4065 844.3198
Agent2|Agent2|mypos|943.4065 844.3198
Agent2|Agent3|mypos|943.4065 844.3198
Agent3|Agent1|mypos|35.0589 844.0807
Agent3|Agent2|mypos|35.0589 844.0807
Agent3|Agent3|mypos|35.0589 844.0807
Agent1|Agent2|reached|yes
Agent1|Agent1|mypos|945.0151 804.9003
Agent1|Agent2|mypos|945.0151 804.9003
Agent1|Agent3|mypos|945.0151 804.9003
Agent2|Agent3|reached|yes
Agent2|Agent1|mypos|74.2896 846.2802
Agent2|Agent2|mypos|74.2896 846.2802
Agent2|Agent3|mypos|74.2896 846.2802
```

Пример файла conections.txt

# Алгоритм нахождения пути

- **Алгоритм навигации** построен на комбинации глобального планирования и локального управления для мобильных роботов в среде с полигонами препятствий и барьерами.
- Каждый агент работает следующим образом: когда агент получает целевую точку, он строит граф видимости, находит кратчайший путь алгоритмом Дейкстры, интерполирует его для плавности, а затем последовательно движется по точкам маршрута, используя пропорциональный регулятор для управления скоростью и поворотом..



# Граф видимости

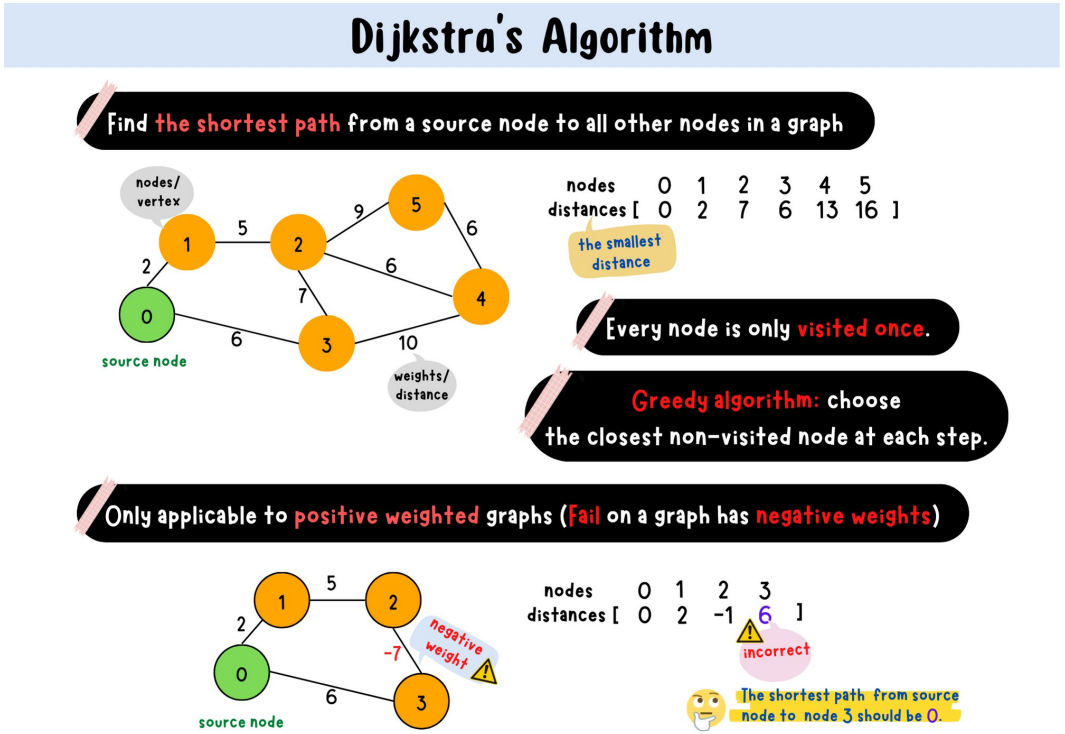
- **Граф видимости** — это структура данных, где узлами являются значимые точки пространства (старт, цель, вершины препятствий), а ребра соединяют только те пары узлов, между которыми можно провести прямую линию, не пересекающую препятствия. После построения такого графа задача поиска кратчайшего пути сводится к классической задаче на графах.
- **Цель построения графа:** найти кратчайший путь от стартовой позиции агента до целевой точки, гарантированно свободный от столкновений. В отличие от сетки (grid-based) методов, граф видимости даёт **оптимальный** (по длине) путь и естественным образом учитывает непрерывное пространство.

# Алгоритм Дейкстры

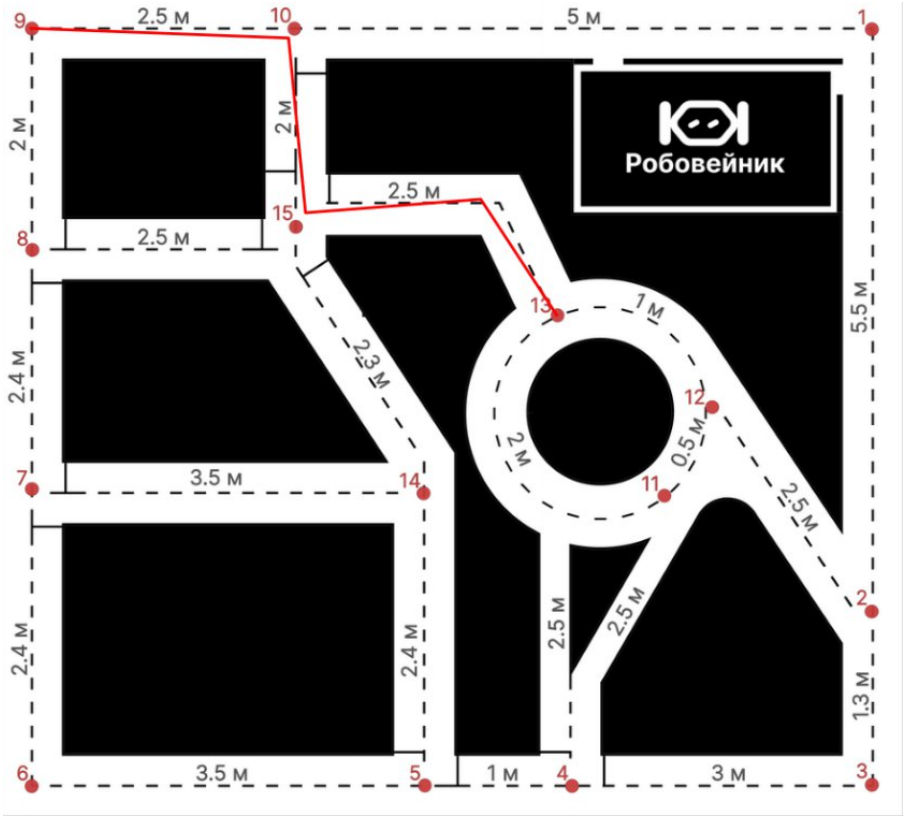
После построения графа применяется классический алгоритм Дейкстры:

- Инициализация: расстояние до старта = 0, до всех остальных узлов = бесконечность
- Пока есть непосещенные узлы:
  - Выбирается непосещенный узел с минимальным расстоянием
  - Для каждого его соседа проверяется, можно ли уменьшить расстояние через текущий узел
  - Если да, расстояние обновляется и запоминается предыдущий узел

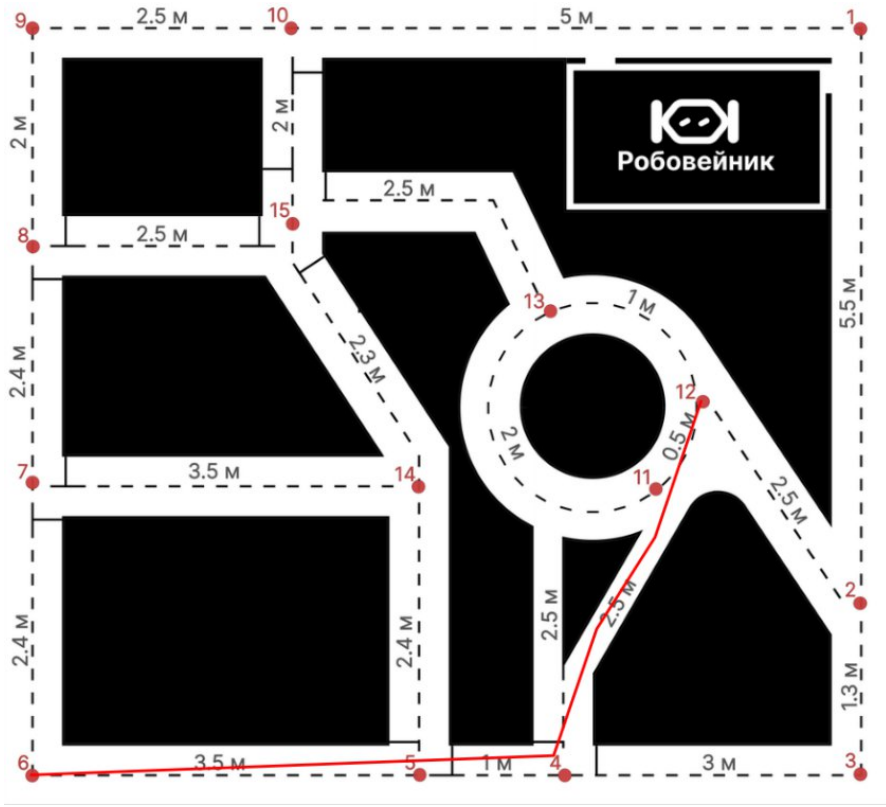
Алгоритм гарантированно находит кратчайший путь в графе с неотрицательными весами ребер.



# Реализация алгоритма Дейкстры




Пример поиска пути от точки 9 к 13



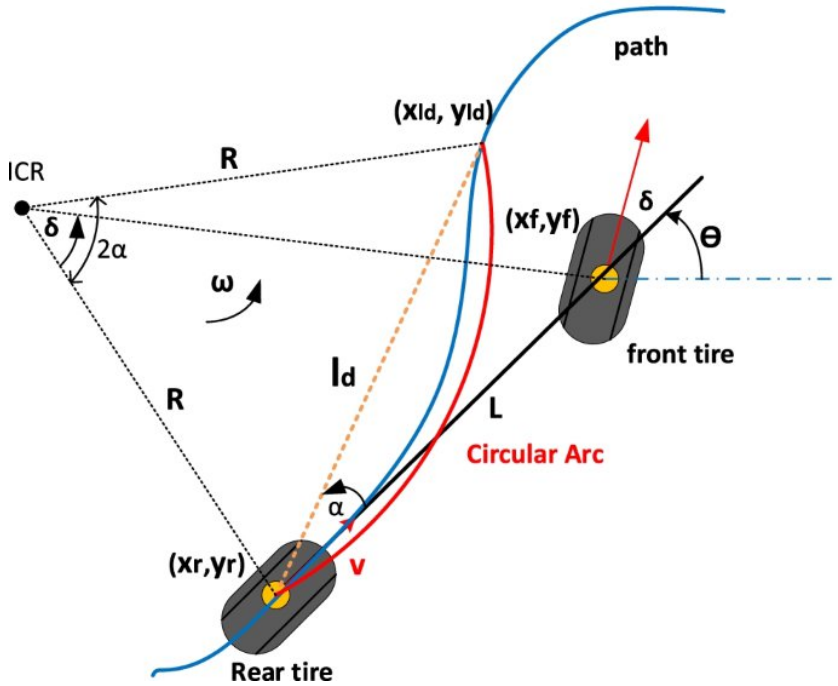

Пример поиска пути от точки 6 к 12

# Интерполяция и регулятор

Полученный путь после применения алгоритма Дейкстры состоит только из опорных точек (узлов графа). Для плавного движения он **интерполируется**:

- Берется первая опорная точка;
- Для каждого следующего отрезка вычисляется направление;
- С шагом step добавляются промежуточные точки;
- Добавляется сама опорная точка.

После интерполяции точки нужно преобразовать в скорости движения и поворота. Используется п**ропорциональный регулятор** — это метод управления, при котором управляющие сигналы (скорость и поворот) вычисляются пропорционально отклонению текущего состояния агента от желаемого. В программах агентов регулятор определяет, как агенту двигаться от текущей позиции к следующей точке маршрута, учитывая угол поворота и расстояние до цели.




Пропорциональный регулятор схематично

# О команде

Все участники “Командная работа” являются студентами физико-технического факультета Кубанского государственного университета:

- Томашевский Никита Александрович – капитан команды, студент 4 курса направления “Информационные системы и технологии”;
- Громов Сергей Валерьевич – студент 1 курса направления “Информационные системы и технологии”;
- Елькин Виктор Андреевич – студент 1 курса направления “Информационные системы и технологии”;
- Лиморова Татьяна Юрьевна – студент 1 курса направления “Информационные системы и технологии”.

КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

ФИЗИКО-ТЕХНИЧЕСКИЙ ФАКУЛЬТЕТ · FACULTY OF PHYSICS AND TECHNOLOGY · ФТФ КубГУ

# Спасибо за внимание!

Команда “Командняяработа”